

Retail Customer Behaviour Analytics & Multi-Horizon Demand Forecasting System

AI/ML Internship Program — Week 3 Industry Mini-Project

GitHub Repository: [anshikakumawat/TechyGirl-week3-retail-analytics](#)

Group Name: TechyGirl
Branch & Year : MCA 2nd year
Submission Date: 05-07-2026

#	Name	Role / Task Assigned
1	Anshika Kumawat	Data & Pipeline — cleaning, EDA, RFM, feature engineering, ColumnTransformer + Pipeline, final notebook merge
2	Samridhi	Customer Segmentation — PCA, K-Means, elbow curve, silhouette score, cluster profiling
3	Khushi Rani	Anomaly Detection & Forecasting — Isolation Forest, decomposition, ACF, ARIMA, Prophet, executive report
4	Shivani Kumari	Documentation & Integration — business insights, contribution log, final ZIP

1. The Problem We Set Out to Solve

Picture a mid-size online retailer. Every day, thousands of orders flow through its systems — but nobody inside the company has a clear, structured view of who its customers really are or what's coming next. Marketing keeps sending the same broad discount email to everyone, whether they're a loyal weekly shopper or someone who bought once and vanished a year ago — and unsurprisingly, most people just ignore it. Meanwhile, the warehouse team is constantly guessing: they over-stock items nobody wants and run out of the ones that are actually flying off the shelves, because they have no reliable short-term demand signal to plan against.

Our job, as a four-person AI/ML data engineering team, was to fix both problems using the same underlying data. We used the UCI Online Retail dataset — real order history from a UK-based online retailer spanning December 2010 to December 2011 — and built three connected systems: a clean customer profile for every shopper, a set of genuine behavioural segments, and a short-term demand forecast. The rest of this document walks through exactly how each of those was built, in the order we built them.

2. Cleaning the Raw Data

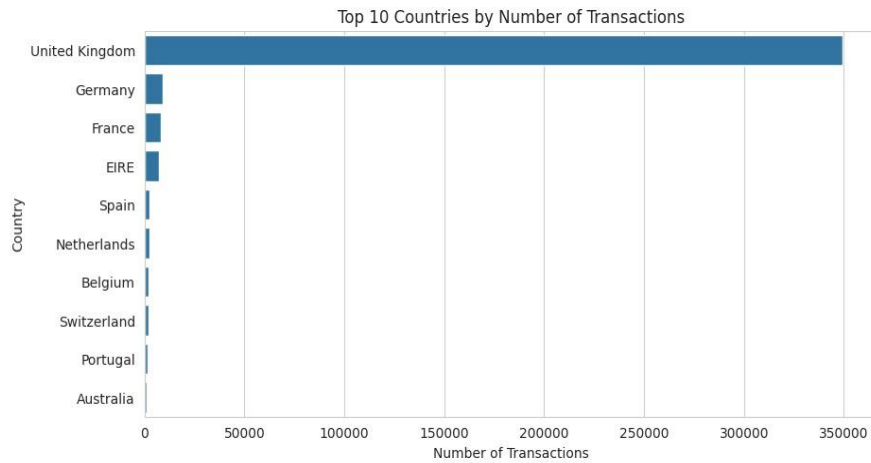
Every real dataset starts out messy, and this one was no exception. The raw file contained 541,909 individual line items — but a lot of those rows couldn't be trusted for customer-level analysis. Some had no CustomerID at all, so there was no way to know who placed that order. Some were exact duplicates. Some were cancelled orders, marked by an InvoiceNo starting with the letter 'C'. And some had a negative or zero Quantity or UnitPrice — usually returns or typing mistakes rather than genuine purchases.

We removed all of these systematically, and for every remaining line item we calculated a TotalPrice field (Quantity multiplied by UnitPrice) so every downstream monetary calculation would have a ready-made number to work with. Here's what that cleaning pass actually did to the size of the dataset:

Stage	Row Count	What changed
Raw dataset	541,909	Every line item, untouched
After cleaning	392,692	Missing IDs, duplicates, cancellations & invalid rows removed
Unique customers retained	4,338	Distinct shoppers left after cleaning

2.1 Looking at the Clean Data

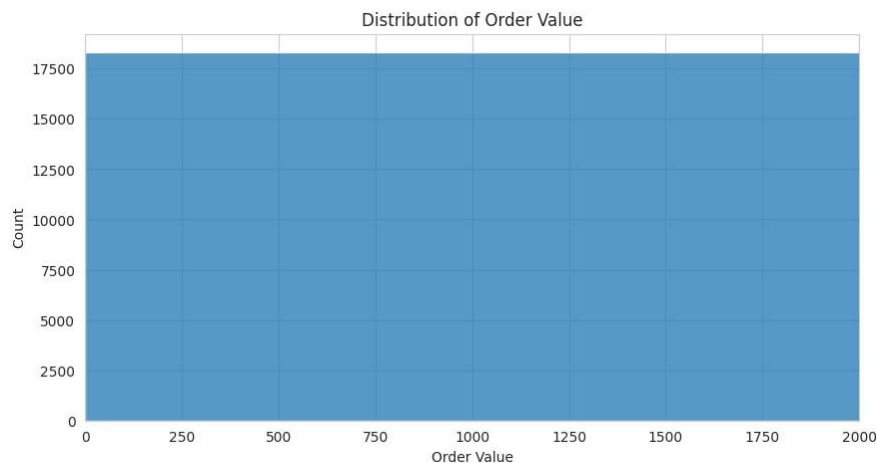
Before building anything predictive, we simply looked at the data — where orders were coming from, how revenue moved through the year, and how big a 'typical' order actually is.



Output 1: Top 10 countries by number of transactions — the United Kingdom dominates, as expected for a UK-based retailer, with Germany, France, EIRE and Spain making up smaller but real shares.



Output 2: Total revenue by month — a clear seasonal build-up toward the year-end holiday period, a pattern we'll see again later in the demand forecast.

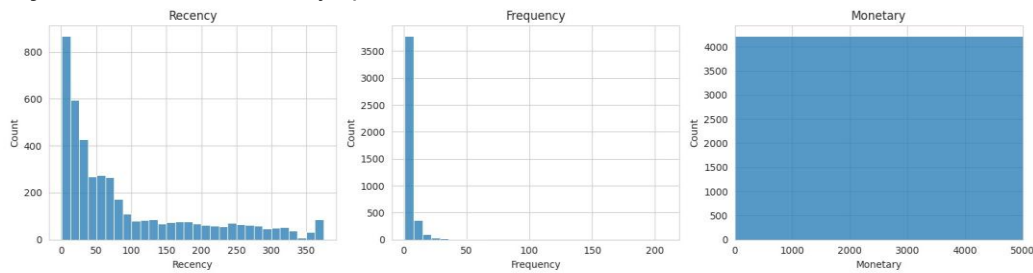


Output 3: Distribution of order value per invoice — most orders are modest in size, but a long right tail of unusually large orders hints that a few bulk or wholesale buyers are hiding in the data.

3. Turning Transactions Into Customer Profiles (RFM)

A pile of transactions doesn't tell a story on its own — we needed to summarise each customer's behaviour into something a human could reason about. That's what RFM analysis does: it boils every customer down to three numbers.

- **Recency** — how many days ago did this customer last buy something?
- **Frequency** — how many separate orders have they placed?
- **Monetary** — how much have they spent with us in total?



Output 4: Distribution of Recency, Frequency and Monetary values across all 4,338 customers. Recency skews toward recently-active shoppers with a long tail of lapsed ones; Frequency and Monetary are both heavily right-skewed — a small group of customers is responsible for a disproportionate share of both orders and revenue, a very typical retail pattern.

4. Adding Behavioural Fingerprints (Feature Engineering)

RFM gives a good starting sketch, but it misses rhythm and geography. So on top of the three RFM numbers, we engineered three more features for every customer:

- **AvgDaysBetweenOrders** — the average number of days between a customer's successive orders, which captures whether someone shops like clockwork or sporadically.
- **WeekendRatio** — what share of a customer's orders land on a weekend, capturing a day-of-week preference.
- **CountryEncoded** — each customer's country, converted into a number using the average order value seen from that country, so a categorical field becomes something a distance-based algorithm can actually use.

Merged together, RFM plus these three engineered features gave us a final table of 4,338 customers by 7 features — one clean, numeric row per shopper, ready for the next stage.

5. Building a Reusable Preprocessing Pipeline

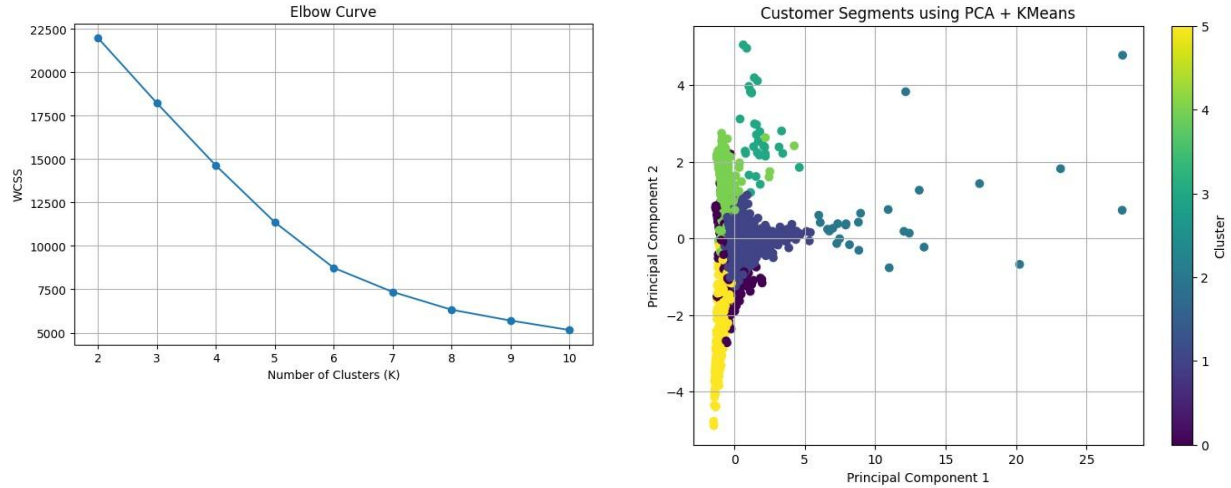
Rather than transforming this feature table by hand — and having to redo that work every time new data comes in — we wrapped the whole preprocessing step inside a scikit-learn Pipeline built around a ColumnTransformer. It does two things automatically: it fills in any missing values using the median (robust to outliers, and necessary because a customer with just one order has no defined 'gap between orders'), and it scales every feature with StandardScaler so that features living on very different scales — Monetary in rupees vs. WeekendRatio between 0 and 1 — all contribute fairly once we start measuring distances between customers.

6. Grouping Customers Into Segments

With a clean, scaled feature matrix in hand, the next question was: can we find natural groups of similar customers hiding inside this data? To answer that, we first used PCA (Principal Component Analysis) to

compress the seven features down to two — purely so we could actually see the data on a 2D scatter plot. Those two components captured about 28.7% and 18.4% of the total variation respectively, enough to see real separation between groups.

We then asked K-Means to try grouping the customers into anywhere from 2 to 10 clusters, tracking how 'tight' each grouping was (technically, the within-cluster sum of squares, or WCSS) at every value of K. Plotting that gives the classic elbow curve — the point where adding more clusters stops helping much:



Output 5 (left): Elbow curve across K = 2–10 — the bend around K = 6 tells us that's a good balance between simplicity and capturing real structure. Output 6 (right): the final six customer segments, visualised on the two PCA components.

We went with K = 6, and to make sure those six groups were meaningfully separated (and not just an arbitrary split), we computed a silhouette score — a number between -1 and 1 that measures how well-matched each customer is to its own cluster versus neighbouring ones. We got **0.447**, a solid result for real-world, noisy behavioural data like this.

The real payoff came from averaging every feature within each cluster, which turned six anonymous numbers into six recognisable customer personas that marketing can actually design campaigns around:

Cluster	Persona	Behavioural Signature
2	High-Value Loyalists	Recent (6 days), extremely frequent (66 orders), and by far the highest average spend (Rs. 85,826) — the core VIP base.
3	Premium International Buyers	Moderate recency and frequency, but a markedly higher CountryEncoded value (109.6) — concentrated in higher-order-value overseas markets.
1	Steady Regular Shoppers	Reasonably recent (40 days), moderate frequency (5.2 orders), healthy spend (Rs. 2,073) — the dependable middle of the customer base.
0	Weekend Bargain Shoppers	A distinctive weekend-heavy buying pattern (85% of orders on weekends), moderate recency, lower average spend.

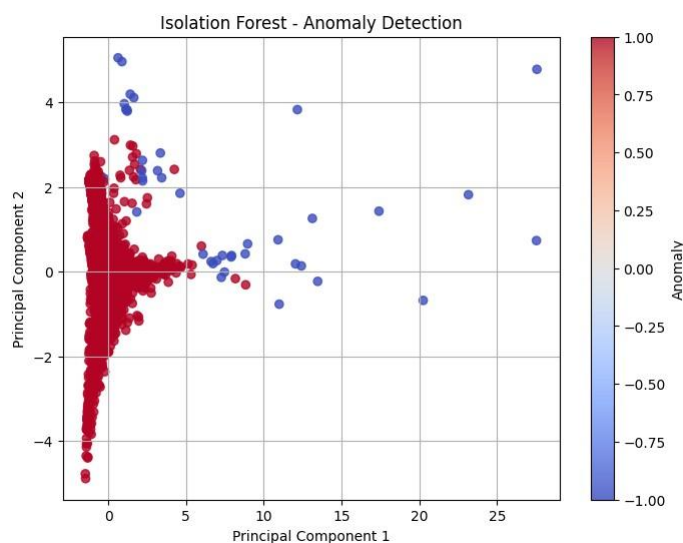
Cluster	Persona	Behavioural Signature
5	Slow-Cycle Shoppers	A long average gap between orders (207 days) with modest spend — they buy rarely, but haven't fully disengaged.
4	Churn-Risk Lapsed Customers	The highest recency (251 days since last order), lowest frequency (1.6), and lowest spend — customers drifting away from the brand.

7. Catching the Outliers Before They Distort Everything

That long right tail we spotted earlier in the order-value distribution wasn't just a curiosity — left unchecked, a handful of huge bulk or wholesale orders can quietly drag cluster centroids and demand forecasts off-course. So we ran an Isolation Forest — a model that isolates unusual points by how easily they can be separated from the rest of the data — at two different levels of the project.

7.1 Customer-Level Anomalies

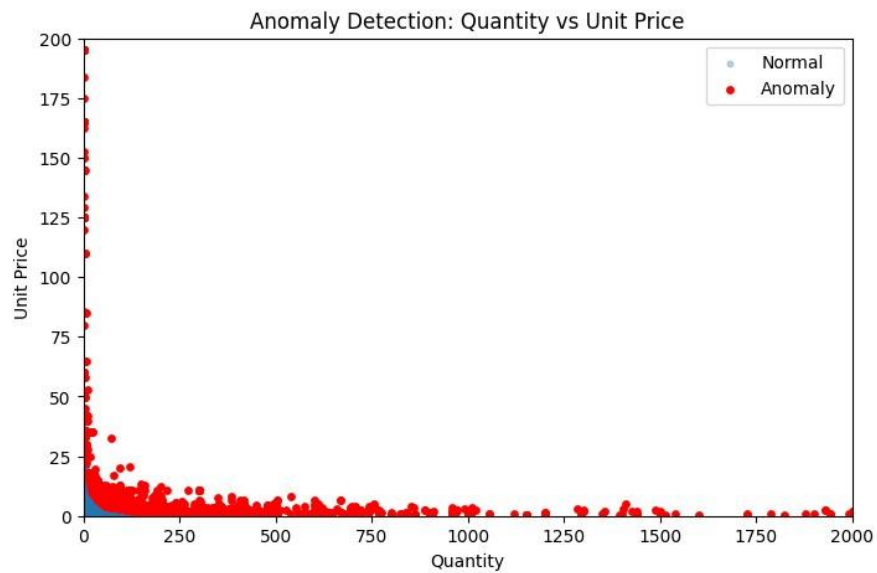
On the scaled customer feature matrix, configured to flag the most unusual 1% of customers, the Isolation Forest identified 44 out of 4,338 customers whose overall RFM/behavioural profile looks nothing like a typical shopper — think extremely high-frequency, high-spend accounts that behave more like a small business than a retail customer. These were kept aside so they didn't pull the six cluster centroids off-centre.



Output 7: Customer-level anomaly flags shown on the same PCA-projected customer space — the scattered points far from the main cloud are the 44 flagged accounts.

7.2 Transaction-Level Anomalies

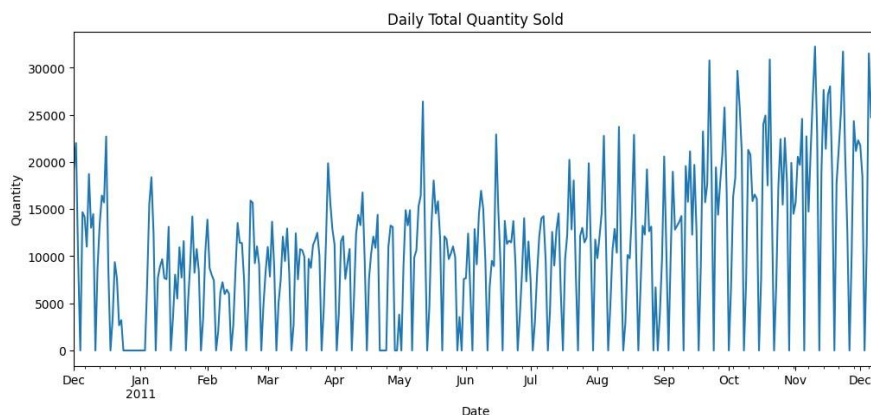
Separately, before building the day-by-day demand signal, we ran a second Isolation Forest directly on individual transactions — this time looking at Quantity, UnitPrice and TotalPrice together, again flagging the most unusual 1%. This caught 3,922 of the 392,692 cleaned transactions, including single orders with quantities as high as 80,995 units and values up to Rs. 168,469.60 — numbers that are simply not part of a normal shopping basket.



Output 8: Transaction-level anomaly detection — Quantity vs Unit Price, with red points marking the flagged, unusually large transactions. These were removed before the demand series was built, so the forecast reflects genuine day-to-day customer buying rather than bulk spikes.

8. Forecasting What Happens Next

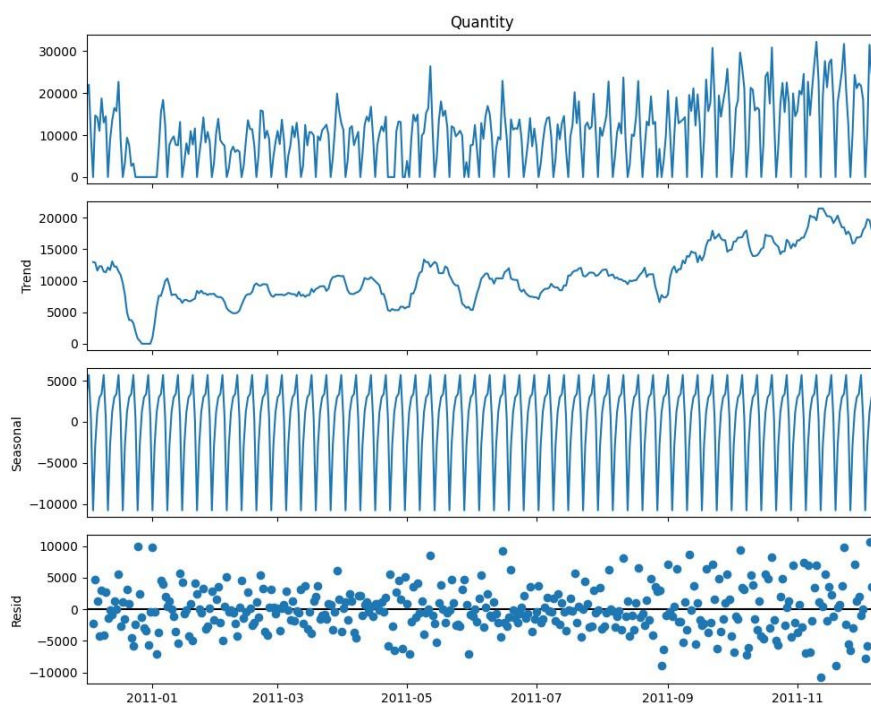
With anomalies out of the way, we collapsed the remaining transactions into a single, continuous daily demand series — simply summing total units sold per calendar day, and filling in zero for any day with no sales, so the timeline never has gaps. That gave us a clean 374-day series to work with.



Output 9: Daily total quantity sold across the full 374-day observation window.

8.1 Breaking the Series Into Trend, Seasonality and Noise

To understand what's actually driving that daily wobble, we ran an additive decomposition with a 7-day period (to capture the weekly retail cycle), splitting the series into three layers: a Trend line showing the big-picture direction of demand, a Seasonal component showing the repeating weekly pattern, and a Residual layer — whatever's left over once you remove the first two.

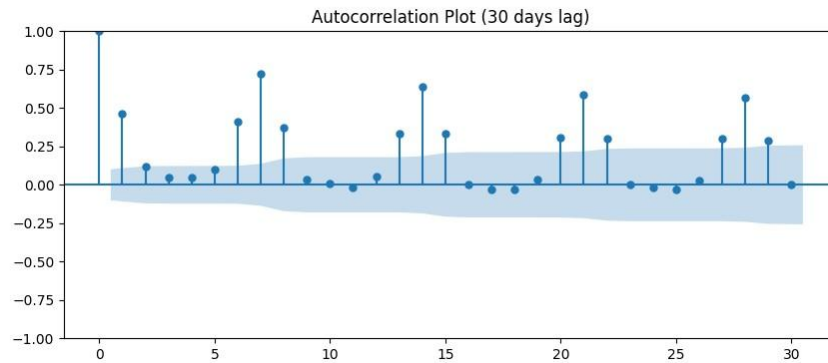


Output 10: Additive decomposition — Trend (overall demand fluctuates with a pull toward the year-end shopping season),

Seasonality (a clear repeating weekly cycle), and Residual (small and centred around zero, meaning trend and seasonality already explain most of the pattern).

We also plotted the Autocorrelation Function (ACF) — essentially asking, 'how much does today's sales figure resemble sales from N days ago?' This confirmed strong short-lag autocorrelation (recent days really do predict

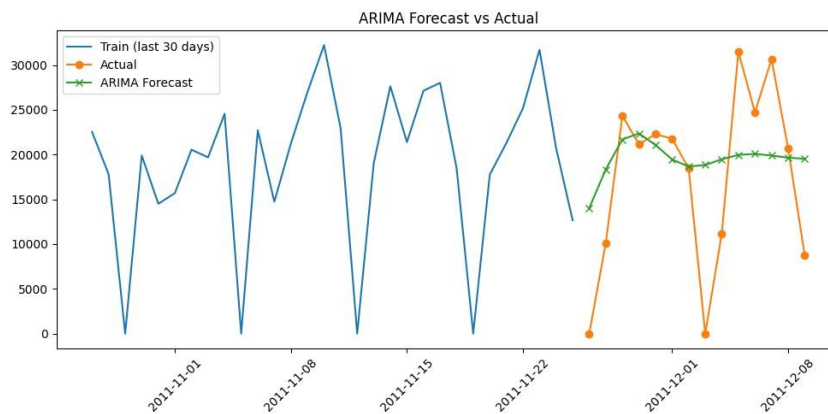
near-term demand) and a recurring spike near multiples of 7 lags, reinforcing the same weekly rhythm we saw in the decomposition — and directly informing how we set up the ARIMA model next.



Output 11: Autocorrelation plot (30-day lag).

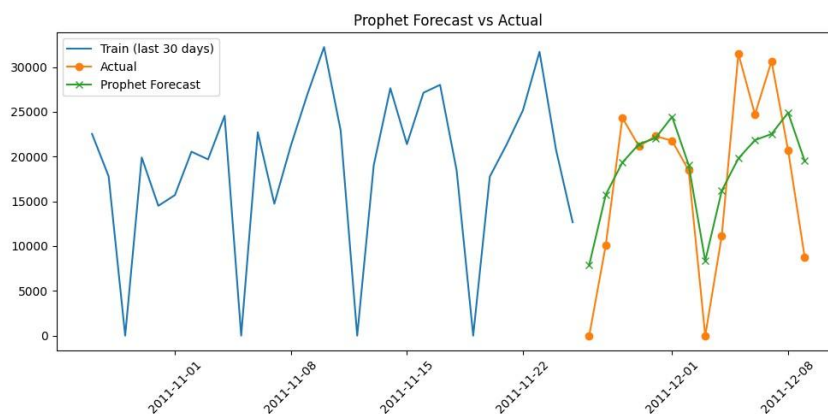
8.2 Two Competing Forecasters: ARIMA vs Prophet

We held out the final 14 days of the series as a test set, and trained two different forecasting approaches on everything before that. The first, $ARIMA(2,1,2)$, is a classical statistical model — 2 autoregressive terms, first-order differencing to remove trend, and 2 moving-average terms, chosen based on what the ACF plot told us.



Output 12: ARIMA forecast vs. actual demand over the 14-day test horizon.

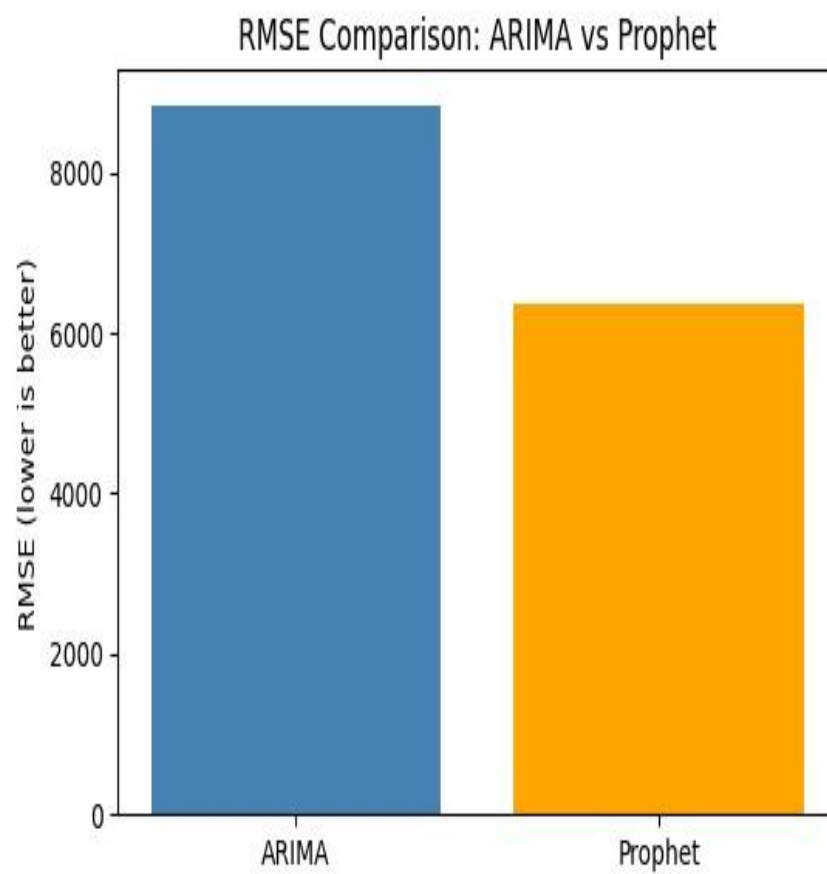
The second, Facebook Prophet, takes a different approach — it automatically detects trend changes and seasonal structure without needing us to manually specify orders, which makes it a useful complement to ARIMA's more rigid, manually-tuned structure.



Output 13: Prophet forecast vs. actual demand over the same 14-day test horizon.

So which one actually won? We scored both with RMSE (Root Mean Squared Error, which penalises big misses more heavily) and MAPE (Mean Absolute Percentage Error, easier to interpret as a percentage), excluding a handful of zero-sales test days from the MAPE calculation to avoid dividing by zero:

Model	RMSE	MAPE (%)
ARIMA	8,829.93	33.98%
Prophet	6,361.81	29.74%



Output 14: RMSE comparison — ARIMA vs Prophet, side by side.

Prophet came out on top on both measures — a lower RMSE (6,361.8 vs 8,829.9) and a lower MAPE (29.7% vs 34.0%) — suggesting its automatic handling of trend and weekly seasonality fits this particular demand pattern slightly better than the fixed-order ARIMA model we specified.

9. So What Should the Business Actually Do?

All of this modelling only matters if it changes what marketing and inventory teams do on Monday morning. Here's how we'd turn these results into action:

- **Stop the one-size-fits-all campaign.** Send retention offers and re-engagement nudges to the Churn-Risk Lapsed and Slow-Cycle segments, while giving High-Value Loyalists early access or loyalty perks instead of the same generic discount everyone else gets.
- **Quarantine the outliers before they reach planning systems.** Route the ~1% of flagged bulk/wholesale customers and transactions to a separate manual review queue, so they never quietly distort reorder thresholds meant for the core retail customer base.
- **Plan inventory around the real weekly rhythm, not a flat schedule.** Use Prophet's 14-day forecast as the primary signal for short-term replenishment, and shift heavier restocking to align with the higher-demand weekdays the decomposition revealed.

10. Wrapping Up

Start to finish, this project turned 541,909 raw, noisy transaction records into something genuinely usable: a clean 4,338-customer feature table, six interpretable personas backed by a silhouette score of 0.447, an anomaly layer that keeps outliers from distorting either the clusters or the demand curve, and a 14-day forecast where Prophet beats ARIMA on both RMSE and MAPE. Marketing walks away with a real basis for targeted campaigns, and inventory walks away with a defensible, data-backed forecast to plan against — which was exactly the gap we set out to close at the start of this document.

11. Who Did What

Member	Contribution
Anshika Kumawat	Data cleaning, EDA, RFM, feature engineering, ColumnTransformer + Pipeline, and merging all module notebooks into the final unified notebook
Samridhi	PCA, K-Means, elbow curve, silhouette score, cluster scatter plot and per-cluster persona profiling
Khushi Rani	Isolation Forest (both levels), time-series preprocessing, decomposition, ACF, ARIMA, Prophet, RMSE/MAPE evaluation, and writing the executive analytical report
Shivani Kumari	Translating model outputs into business insights, compiling the contribution log, and assembling the final submission ZIP

Appendix: Key Python Code

A look under the hood — representative code from each stage of the pipeline described above. Full, executable notebooks are available in the GitHub repository linked on the cover page.

A. Data Cleaning

```
# Data cleaning df = df.dropna(subset=["CustomerID"])
df = df.drop_duplicates() df =
df[~df["InvoiceNo"].astype(str).str.startswith("C")] df
= df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]
df["TotalPrice"] = df["Quantity"] * df["UnitPrice"]
```

B. RFM Feature Computation

```
# RFM feature computation snapshot_date = df["InvoiceDate"].max() +
pd.Timedelta(days=1) rfm = df.groupby("CustomerID").agg(
Recency=("InvoiceDate", lambda x: (snapshot_date - x.max()).days),
Frequency=("InvoiceNo", "nunique"),
Monetary=("TotalPrice", "sum"),
)
```

C. ColumnTransformer + Pipeline

```
# ColumnTransformer + Pipeline from
sklearn.pipeline import Pipeline from
sklearn.compose import ColumnTransformer from
sklearn.impute import SimpleImputer from
sklearn.preprocessing import StandardScaler

numeric_pipeline = Pipeline([ ("imputer",
SimpleImputer(strategy="median")),
("scaler", StandardScaler()),
])
preprocessor = ColumnTransformer([("num", numeric_pipeline, numeric_cols)])
X_scaled = preprocessor.fit_transform(X)
```

D. PCA + K-Means Clustering

```
# PCA + K-Means clustering (from
customer_clustering.ipynb) pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

wcss = [] for k in range(2, 11): model =
KMeans(n_clusters=k, random_state=42, n_init=10)
model.fit(X_scaled) wcss.append(model.inertia_)

kmeans = KMeans(n_clusters=6, random_state=42,
n_init=10) df["Cluster"] = kmeans.fit_predict(X_scaled)
score = silhouette_score(X_scaled, df["Cluster"])
```

E. Isolation Forest Anomaly Detection

```
# Isolation Forest anomaly detection (from
customer_clustering.ipynb) iso =
IsolationForest(contamination=0.01, random_state=42) df["Anomaly"]
= iso.fit_predict(X_scaled) anomalies = df[df["Anomaly"] == -1]
```

F. ARIMA vs Prophet Forecasting

```
# ARIMA vs Prophet forecasting from
statsmodels.tsa.arima.model import ARIMA
from prophet import Prophet

arima_model = ARIMA(train, order=(2, 1, 2)).fit()
arima_forecast = arima_model.forecast(steps=14)

prophet_df = train.reset_index().rename(columns={"index": "ds", "Quantity":
"y"}) prophet_model = Prophet().fit(prophet_df) future =
prophet_model.make_future_dataframe(periods=14) prophet_forecast =
prophet_model.predict(future)
```